

CensusConnect User Authentication System

BEREA COLLEGE

Nicholas Hamilton

Tech Stack: Python/PostgreSQL/bcrypt | Authentication & Authorization | Security: OWASP Compliance & Multi-Layer Protection

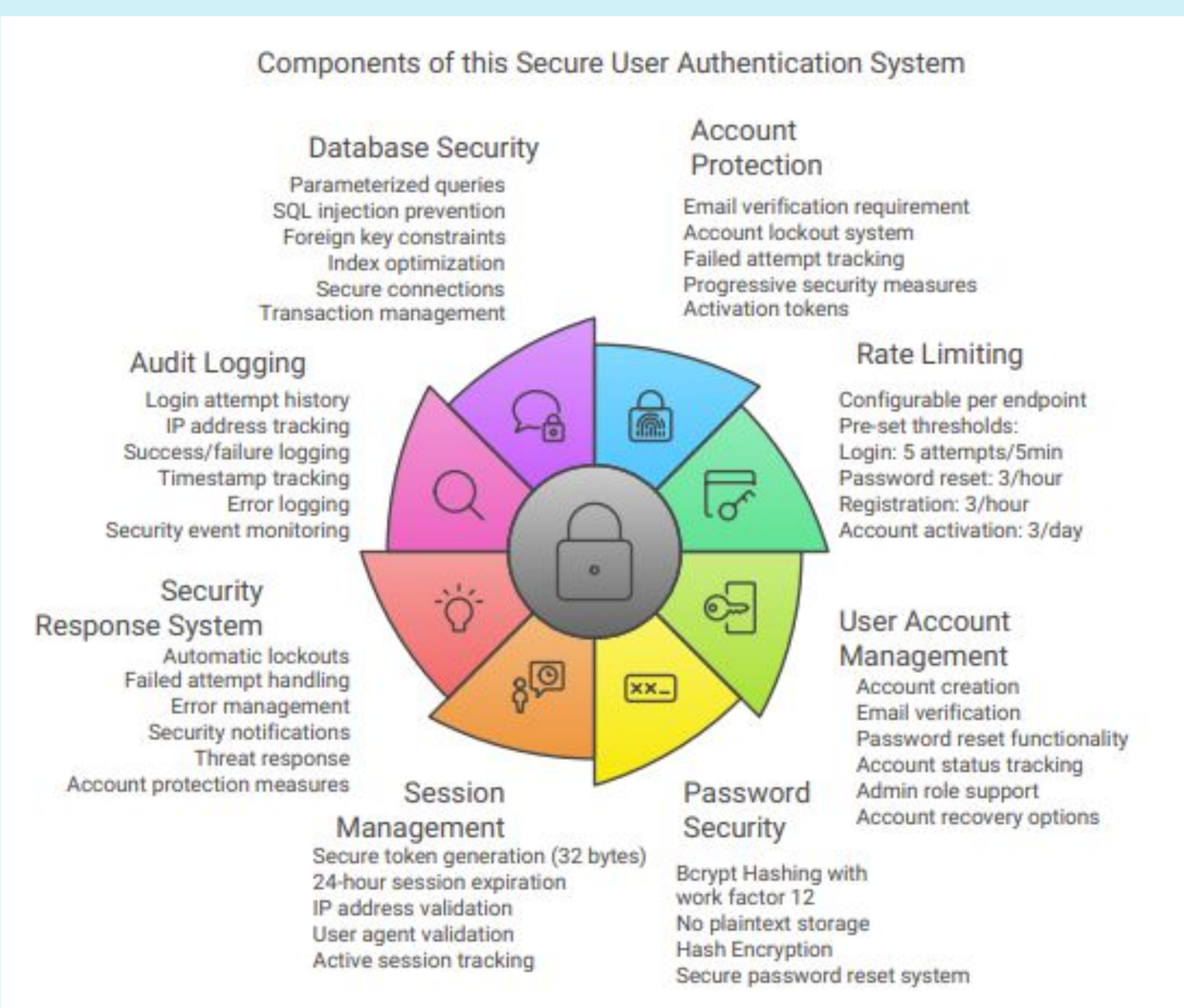
Introduction & Objectives

CensusConnect

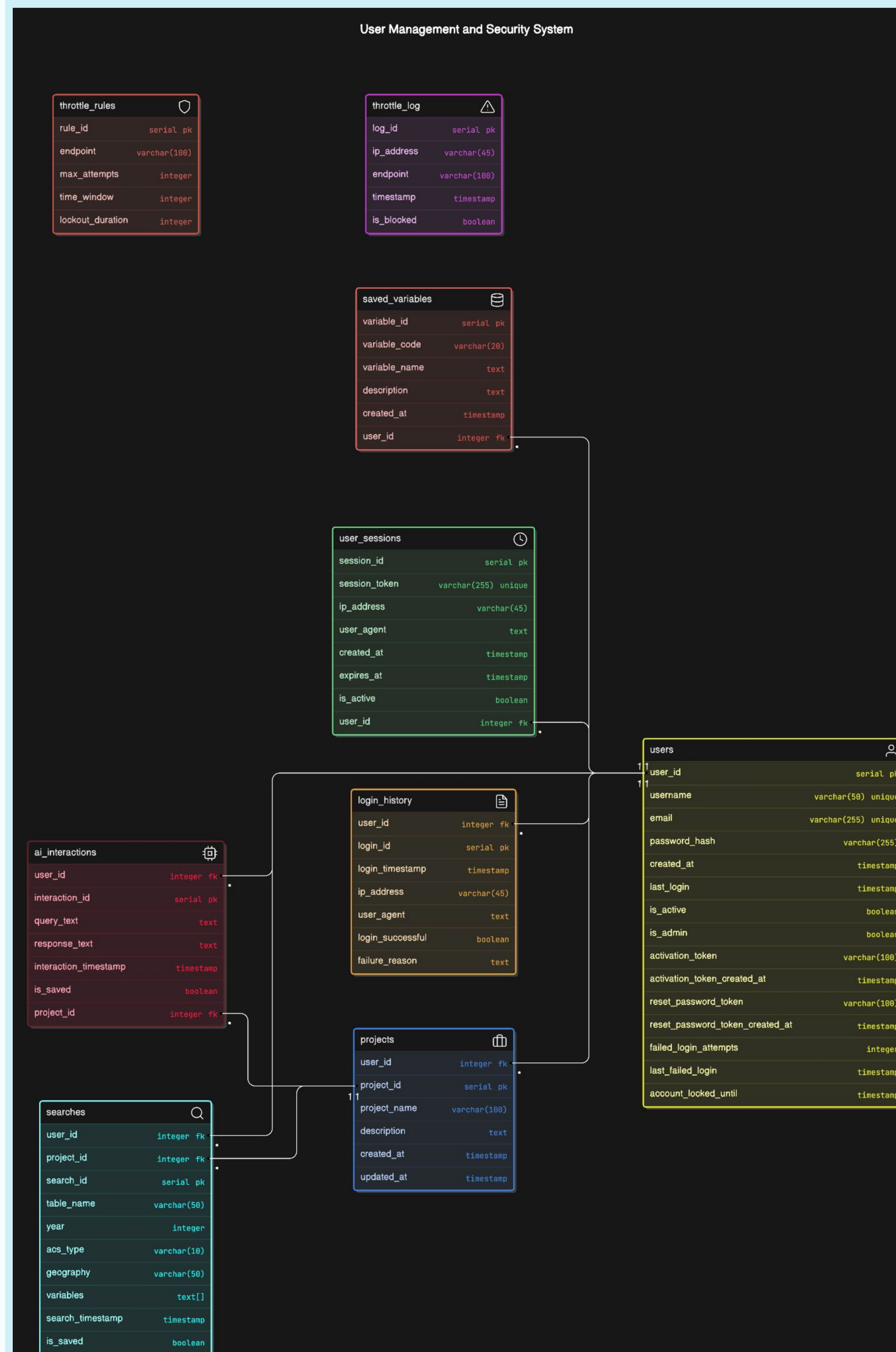
Is an application centered around democratizing data access by eliminating some of the potential areas of mistakes in collecting data from the US American Community Survey, while eliminating the need for coding and in depth ACS knowledge.

User Authentication Overview

- Purpose: Enterprise-grade authentication system for ACS Data Application
- Target Users: Researchers, analysts, and public users accessing ACS data
- Protection: Defense in Depth, with several layers of protection guarding user data



System Architecture



Security Threats & Defense Mechanisms

Brute Force Protection - Repeated login attempts

```
# Automatic account lockout after 5 failed attempts
cur.execute("""
    UPDATE users
    SET failed_login_attempts = failed_login_attempts + 1,
        account_locked_until = CASE
            WHEN failed_login_attempts + 1 >= 5
            THEN CURRENT_TIMESTAMP + interval '15 minutes'
            ELSE account_locked_until
        END
    WHERE user_id = %s
""")
```

Password Security

```
# Strong password hashing with bcrypt
password_hash = bcrypt.hashpw(
    password.encode('utf-8'),
    bcrypt.gensalt(rounds=12)
)

# Database storage with no plaintext
password_hash VARCHAR(255) NOT NULL,
reset_password_token VARCHAR(100),
reset_password_token_created_at TIMESTAMP
```

Session Hijacking

```
# Secure session validation
cur.execute("""
    SELECT user_id FROM user_sessions
    WHERE session_token = %s
    AND ip_address = %s
    AND user_agent = %s
    AND expires_at > CURRENT_TIMESTAMP
    AND is_active = TRUE
""")

# Session table structure
CREATE TABLE user_sessions (
    session_token VARCHAR(255) UNIQUE NOT NULL,
    ip_address VARCHAR(45),
    user_agent TEXT,
    expires_at TIMESTAMP WITH TIME ZONE NOT NULL
);
```

SQL Injection

```
# Parameterized queries throughout
cur.execute("""
    SELECT * FROM users
    WHERE username = %s
""", (username,))

# Proper indexing
CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_email ON users(email);
```

Rate Limiting

```
# Endpoint-specific rate limiting
INSERT INTO throttle_rules
(endpoint, max_attempts, time_window, lockout_duration)
VALUES
('login', 5, 300, 900),      -- 5 attempts/5min
('password_reset', 3, 3600, 7200), -- 3/hour
('register', 3, 3600, 7200);   -- 3/hour

# Rate limit checking
cur.execute("""
    SELECT COUNT(*) FROM throttle_log
    WHERE ip_address = %s
    AND endpoint = %s
    AND timestamp > CURRENT_TIMESTAMP -
        interval '1 second' * %s
""")
```

Resources

- Authentication cheat sheet¶. Authentication - OWASP Cheat Sheet Series. (n.d.). https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Password storage cheat sheet¶. Password Storage - OWASP Cheat Sheet Series. (n.d.). https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Simpson, J. (2023, September 29). Everything you need to know about API rate limiting: Nordic apis | Nordic APIs. <https://nordicapis.com/everything-you-need-to-know-about-api-rate-limiting/>
- SQL Injection. SQL Injection | OWASP Foundation. (n.d.). https://owasp.org/www-community/attacks/SQL_injection
- Tang, A. (2024, August 28). How to protect your flask applications | escape blog. Escape. <https://escape.tech/blog/best-practices-protect-flask-applications/>
- The Ultimate Guide to User Authentication. (n.d.). <https://blog.datachef.co/user-authentication?x-host=blog.datachef.co>